

Secure Order Processing Microservice

Role: Principal Solutions Architect

Difficulty: junior

Project Type: backend

Duration: 2 days

Focus: security

Candidate: Siddharth Rao

Project Overview

Build a practical, measurable backend project that leverages the candidate's architecture, cloud native, microservices, event driven, and security skills. The project will demonstrate the ability to design a secure, event driven microservice that consumes orders from Kafka, persists them to a database, and exposes a protected REST API. It introduces new learning around JWT authentication, HTTPS, and basic Terraform deployment while remaining realistic for a 2 day sprint.

Technologies

- Python
- Docker
- Kubernetes
- Kafka
- Terraform
- JWT
- HTTPS
- PostgreSQL
- AWS
- EKS
- S3
- RDS

Requirements

- Implement a microservice that consumes order events from Kafka
- Persist order data to PostgreSQL
- Expose a secure REST API to query order status
- Use Docker Compose for local development and Terraform for cloud deployment
- Apply JWT authentication and HTTPS
- Encrypt data at rest in the database

Objectives

- Demonstrate secure microservice design
- Implement event driven processing
- Deploy locally and to AWS EKS
- Show basic Terraform infrastructure as code

Functional Requirements

- Consume messages from Kafka topic 'orders'
- Persist order record with fields id, customer_id, items, status, timestamps
- Expose GET /orders/{id} to return order status
- Require a valid JWT in Authorization header
- Return 401 for unauthenticated requests
- Log all processing steps

Non-Functional Requirements

- API must be served over HTTPS
- Database data encrypted at rest
- Service should handle at least 100 orders/minute
- Deployment scripts must be idempotent
- Code must be documented and follow PEP8

Modules

OrderConsumer

Kafka consumer that processes order events

- Connect to Kafka
- Deserialize JSON
- Validate payload
- Persist to DB

OrderService

Business logic for order status retrieval

- Query DB
- Return DTO
- Handle missing records

AuthMiddleware

JWT validation and request authentication

- Verify token signature
- Extract user claims
- Reject invalid tokens

API

FastAPI application exposing order endpoints

- GET /orders/{id}
- HTTPS support
- Swagger docs

Implementation Plan

Design

- Define data model
- Sketch Kafka topic schema
- Plan Docker/K8s deployment
- Draft Terraform modules

Development

- Implement OrderConsumer in Python
- Set up FastAPI app with AuthMiddleware
- Create PostgreSQL schema
- Write unit tests

Testing

- Run integration tests with local Kafka and Postgres
- Test JWT authentication
- Performance test 100 orders/min

Deployment

- Create Docker Compose file for local dev
- Write Terraform scripts for EKS, RDS, S3
- Deploy to AWS and verify HTTPS

Evaluation Criteria

Correctness of implementation (40%)

All functional requirements are met and the service runs end to end

Security compliance (30%)

JWT authentication, HTTPS, and data encryption are correctly implemented

Code quality and documentation (30%)

Code follows style guidelines, is well documented, and includes tests

Deliverables

- Git repository with source code
- Docker Compose file
- Terraform scripts
- API documentation (OpenAPI)
- Test report

Suggested Folder Structure

- src/

- tests/
- docs/
- infra/
- scripts/

API Endpoints

GET /orders/{id}

Retrieve order status for a given order ID

Database Design

orders

Store order records

- id
- customer_id
- items
- status
- created_at
- updated_at

Testing Plan

Candidate Fit

The project aligns with Siddharth Rao's architecture, cloud native, microservices, and event driven expertise while challenging him to apply security best practices and basic Terraform deployment. It is realistic for a 2 day sprint and provides clear, measurable outcomes.